

Struktury danych	
Kierunek <i>Informatyczne Systemy Automatyki</i>	Termin <i>poniedziałek TN 11¹⁵ – 13⁰⁰</i>
Imię, nazwisko, numer albumu <i>Jan Hulbój 284197, Maciej Polanowski 284195</i>	Data <i>13.04.2026</i>
Link do projektu https://github.com/mlody-jano/dataStructures	



Politechnika
Wrocławska

SPRAWOZDANIE – MINIPROJEKT NR 2

Poniższy dokument jest kompleksowym sprawozdaniem opisującym badania przeprowadzone w ramach pierwszego projektu na przedmiocie **Struktury Danych**. Obiektem pierwszego projektu była samodzielna implementacja **kolejki priorytetowej**, z wykorzystaniem dwóch różnych struktur danych, jednej będącej **kopcem**. Każda ze struktur została zbadana pod kątem złożoności obliczeniowej wybranych operacji, które zostały podane w wymaganiach projektowych.

Spis treści

1 Wstęp teoretyczny	2
1.1 Kolejka jako ADT	2
1.2 Kolejka priorytetowa na tablicy dynamicznej	2
1.3 Kolejka priorytetowa na kopcu	3
2 Założenia projektowe	3
3 Wyniki badań	5
3.1 Operacja <code>enqueue()</code>	5
3.2 Operacja <code>extractMax()</code>	6
3.3 Operacja <code>peek()</code>	7
3.4 Operacja <code>decreaseKey()</code>	8
3.5 Operacja <code>increaseKey()</code>	9
3.6 Operacja <code>size()</code>	10
4 Wnioski	11

Spis rysunków

1 Skrypt generujący dane.	4
2 Fragment kodu odpowiedzialny za pomiar czasu 100 razy.	4
3 Funkcja wywołująca pomiar czasu.	4
4 Zależność średniego czasu wykonania operacji <code>enqueue()</code> od rozmiaru struktury danych.	5
5 Zależność średniego czasu wykonania operacji <code>extractMax()</code> od rozmiaru struktury danych.	6
6 Zależność średniego czasu wykonania operacji <code>peek()</code> od rozmiaru struktury danych.	7
7 Zależność średniego czasu wykonania operacji <code>decreaseKey()</code> od rozmiaru struktury danych.	8
8 Zależność średniego czasu wykonania operacji <code>increaseKey()</code> od rozmiaru struktury danych.	9
9 Zależność średniego czasu wykonania operacji <code>size()</code> od rozmiaru struktury danych.	10

Spis tabel

1	Złożoność obliczeniowa operacji kolejki priorytetowej na posortowanej tablicy dynamicznej	2
2	Złożoność obliczeniowa operacji kolejki priorytetowej implementowanej na kopcu	3
3	Wyniki testów operacji enqueue()	5
4	Wyniki testów operacji extractMax()	6
5	Wyniki testów operacji peek()	7
6	Wyniki testów operacji decreaseKey()	8
7	Wyniki testów operacji increaseKey()	9
8	Wyniki testów operacji size()	10

1 Wstęp teoretyczny

1.1 Kolejka jako ADT

Abstrakcyjny typ danych tj. **ADT** jest tworem ideowym, który pozwala zdefiniować interfejs oraz własności struktury tworzonej, pozostawiając definicję poszczególnych operacji, w gestii odpowiedniej implementacji. Taki proces pozwala stworzyć pewnego rodzaju polimorficzne podejście, w którym wybrane ADT stanowi fundament do tworzenia jego implementacji, w różne, nie zawsze takie same sposoby.

1.2 Kolejka priorytetowa na tablicy dynamicznej

Kolejka priorytetowa zaimplementowana na **posortowanej tablicy dynamicznej** opiera się na odpowiednim wstawianiu par wartość-klucz do tablicy. Operacja **enqueue(item)** sprawdza priorytet wartości kolejkowej, a następnie przechodzi przez tablice, szukając odpowiedniego miejsca do wstawienia. Skutkiem tego jest liniowa ($O(n)$) złożoność obliczeniowa operacji **enqueue()**, lecz pozwala to na zawsze stały ($O(1)$) dostęp do elementu największego, przez metodę **extractMax()** lub **peek()**. Ciekawym miejscem, w którym można przyjąć różne strategie jest sytuacja w której dodajemy do kolejki parę z priorytetem, który jest już obecny w kolejce. W tym projekcie zastosowane zasadę kolejki FIFO, tj. pierwszy kolejkowy element z danym priorytetem jest "wyżej" w kolejce i każde kolejne rekordy z tym samym priorytetem, będą usytuowane za nim.

Rzeczywiste zastosowania:

- Statyczne harmonogramy i systemy zdominowane przez odczyt (Read-heavy): W aplikacjach biznesowych lub systemach logistycznych, gdzie lista priorytetów ładowana jest rzadko (np. raz dziennie), ale bardzo często odpytywana o najważniejszy element, stosuje się posortowaną tablicę dynamiczną. Wstawianie elementu jest wtedy kosztowne (wymaga przesunięcia pamięci w czasie $O(n)$), jednak pobranie zadania o najwyższym priorytecie odbywa się błyskawicznie w czasie stałym $O(1)$.

Oczekiwane złożoności obliczeniowe operacji na kolejce priorytetowej wyglądają następująco:

Tabela 1: Złożoność obliczeniowa operacji kolejki priorytetowej na **posortowanej tablicy dynamicznej**

Lp.	Operation	Best	Average	Worst
1.	enqueue()	$O(1)$	$O(n)$	$O(n)$
2.	extractMax()	$O(1)$	$O(1)$	$O(1)$
3.	peek()	$O(1)$	$O(1)$	$O(1)$
4.	decreaseKey()	$O(1)$	$O(n)$	$O(n)$
5.	increaseKey()	$O(1)$	$O(n)$	$O(n)$
6.	size()	$O(1)$	$O(1)$	$O(1)$

1.3 Kolejka priorytetowa na kopcu

Kolejka priorytetowa zaimplementowana na kopcu binarnym (ang. binary heap) opiera się na strukturze drzewa binarnego spełniającego własność kopca MAX, tj. każdy węzeł ma priorytet nie mniejszy niż priorytety jego dzieci. W praktyce kopiec jest przechowywany jako tablica, w której dla węzła o indeksie i i jego dzieci znajdują się pod indeksami $2i + 1$ oraz $2i + 2$. Operacja `enqueue(item)` wstawia nowy element na końcu tablicy, a następnie wykonuje operację „przesiewania w górę” (`reapirUp`), przywracając własność kopca w czasie $O(\log n)$. Operacja `extractMax()` pobiera element z korzenia (indeks 0), zastępuje go ostatnim elementem tablicy i wykonuje „przesiewanie w dół” (`repairDown`), również w czasie $O(\log n)$. Operacja `peek()` zwraca korzeń kopca w czasie stałym $O(1)$. Operacje `decreaseKey()` oraz `increaseKey()` wymagają uprzedniego zlokalizowania elementu w tablicy, co w standardowej implementacji bez dodatkowej struktury indeksującej zajmuje $O(n)$, a następnie naprawy własności kopca w $O(\log n)$. W projekcie zastosowano tę samą zasadę kolejki FIFO co w przypadku posortowanej tablicy, element wstawiony wcześniej z danym priorytetem jest traktowany jako „wyższy” w kolejce.

Rzeczywiste zastosowania:

- Algorytmy grafowe (Dijkstra, Prim): Stanowią fundament działania nawigacji GPS, protokołów routingu w sieciach komputerowych oraz systemów wyszukiwania ścieżek (pathfinding) w grach wideo. Znajdowanie najkrótszej drogi polega na nieustannym dodawaniu nowych wierzchołków i wyciąganiu tych o najmniejszym koszcie do celu. Kopiec obsługuje obie te operacje w czasie logarytmicznym $O(\log n)$, co pozwala na szybkie przeliczanie tras nawet na mapach zawierających miliony skrzyżowań.

Oczekiwane złożoności obliczeniowe operacji na kolejce priorytetowej wyglądają następująco:

Tabela 2: Złożoność obliczeniowa operacji kolejki priorytetowej implementowanej na **kopcu**

Lp.	Operation	Best	Average	Worst
1.	<code>enqueue()</code>	$O(\log n)$	$O(\log n)$	$O(\log n)$
2.	<code>extractMax()</code>	$O(\log n)$	$O(\log n)$	$O(\log n)$
3.	<code>peek()</code>	$O(1)$	$O(1)$	$O(1)$
4.	<code>decreaseKey()</code>	$O(\log n)$	$O(n)$	$O(n)$
5.	<code>increaseKey()</code>	$O(\log n)$	$O(n)$	$O(n)$
6.	<code>size()</code>	$O(1)$	$O(1)$	$O(1)$

2 Założenia projektowe

W celu zapewnienia odpowiedniego podłoża merytorycznego do wyciągnięcia wniosków dotyczących badanych struktur danych, przyjęte zostały następujące założenia:

- **Rozmiary struktur:** 10 rozmiarów z zakresu 20000 - 20000, skok co 20 tys.
- **Ilość powtórzeń operacji:** 100
- **Sposób generacji danych:** Skrypt Python generujący dane do pliku `.csv`.
- **Sposób pomiaru czasu:** Uśrednienie czasu operacji dla wybranej liczby kopii określonej parametrem COPIES.

Testy utworzonych struktur danych były prowadzone na urządzeniu wyposażonym w następujące komponenty:

- **Procesor** - Apple M4 10 Core CPU 4Ghz
- **RAM** - 16GB

Dane testowe zostały wygenerowane na podstawie następującego skryptu wykonanego w języku Python. Zastosowanie wybranej wartości ziarna podczas generacji pozwala na uzyskanie odtwarzalnych wyników. Poniżej zawarty został fragment kodu odpowiedzialny za generację danych:

```
def generate_test_data(filename, count, seed_value=0):
    random.seed(seed_value)

    min_val = -2147483648
    max_val = 2147483647
    min_prio = 0
    max_prio = 2000000

    print(f"Generowanie {count} rekordów do pliku '{filename}'...")
    print(f"Użyte ziarno (seed): {seed_value}")

    try:
        os.makedirs(os.path.dirname(filename), exist_ok=True)
        with open(filename, 'w', newline='') as f:
            writer = csv.writer(f)
            writer.writerow(['value', 'priority'])
            for _ in range(count):
                writer.writerow([
                    random.randint(min_val, max_val),
                    random.randint(min_prio, max_prio)
                ])
            print("Plik wygenerowany pomyślnie!")
    except IOError as e:
        print(f"Błąd zapisu do pliku: {e}")
```

Rysunek 1: Skrypt generujący dane.

Badanie czasu operacji zostało zrealizowane w postaci uśrednienia czasu wykonania zadanej ilości operacji określonej parametrem COPIES, który w tym przypadku został wybrany na COPIES = 100. Zabieg ten został zrealizowany w celu zapewnienia jednakowych warunków początkowych każdemu powtórzeniu badanej operacji, tym samym, zapewnienia gwarancji poprawności wyników. Fragment kodu odpowiedzialnego za pomiar czasu:

```
double t = measure([&]() {
    for (int i = 0; i < COPIES; i++)
        q[i].enqueue(extraElement);
});
enqTimes.push_back(t / COPIES);
```

Rysunek 2: Fragment kodu odpowiedzialny za pomiar czasu 100 razy.

```
template <typename Func>
double measure(Func&& f) {
    auto t0 = Clock::now();
    f();
    auto t1 = Clock::now();
    return std::chrono::duration_cast<ns>(t1 - t0).count();
}
```

Rysunek 3: Funkcja wywołująca pomiar czasu.

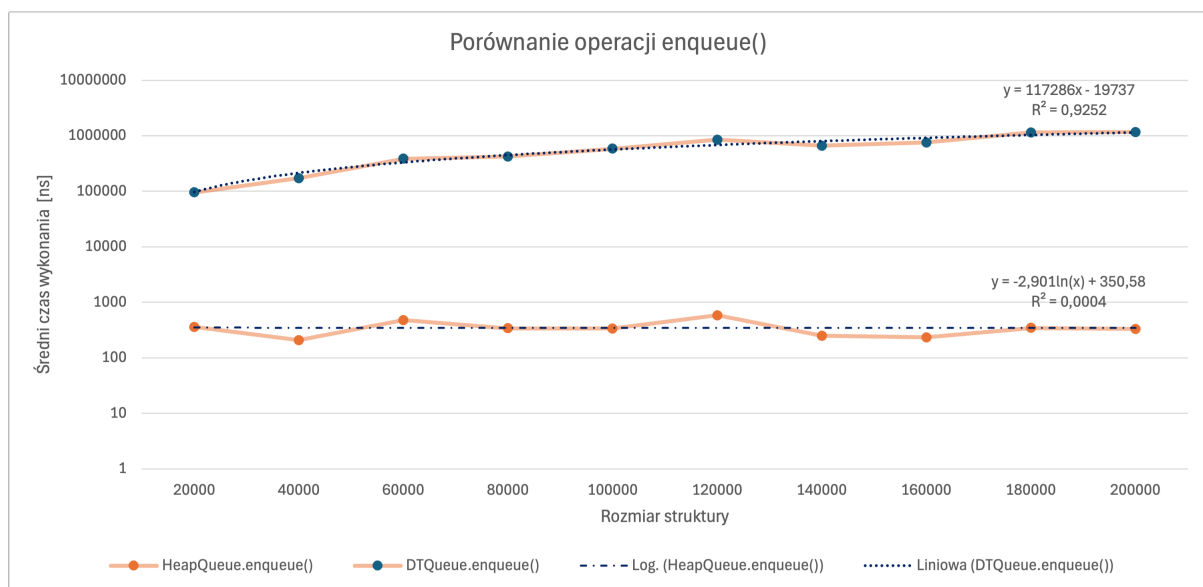
3 Wyniki badań

3.1 Operacja enqueue()

Poniżej zawarte zostało zestawienie złożoności obliczeniowych operacji `enqueue(item)`, dla różnych implementacji kolejki priorytetowej, oraz różnych zestawów danych wejściowych. Wyniki zostały zaprezentowane w postaci tabeli, oraz wykresu:

Tabela 3: Wyniki testów operacji `enqueue()`

Lp.	Size	Operation time [ns]	
		DTQueue	HeapQueue
1.	n = 20000	95410,2	359,1
2.	n = 40000	172159	208,4
3.	n = 60000	385249	478,2
4.	n = 80000	422849	338,2
5.	n = 100000	585896	337,5
6.	n = 120000	851222	580,8
7.	n = 140000	664224	250,2
8.	n = 160000	757953	234,1
9.	n = 180000	1151290	345,8
10.	n = 200000	11671120	329,7



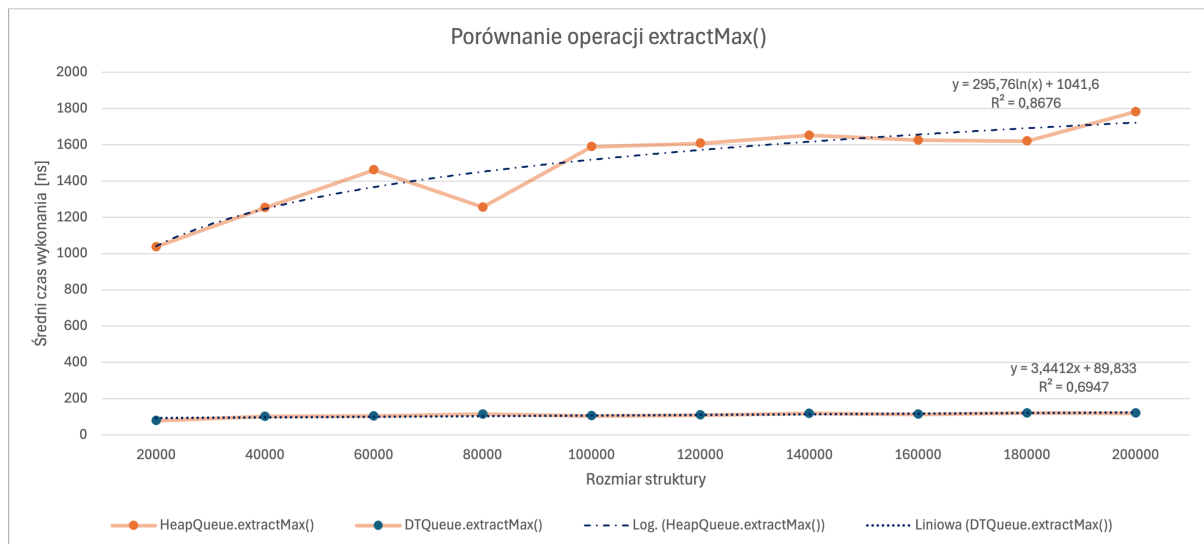
Rysunek 4: Zależność średniego czasu wykonania operacji `enqueue()` od rozmiaru struktury danych.

3.2 Operacja extractMax()

Poniżej zawarte zostało zestawienie złożoności obliczeniowych operacji `extractMax()`, dla różnych implementacji kolejki priorytetowej, oraz różnych zestawów danych wejściowych. Wyniki zostały zaprezentowane w postaci tabeli, oraz wykresu:

Tabela 4: Wyniki testów operacji `extractMax()`

Lp.	Size	Operation time [ns]	
		DTQueue	HeapQueue
1.	n = 20000	78,8	1037,1
2.	n = 40000	101,8	1252,8
3.	n = 60000	104,6	1460,8
4.	n = 80000	114,4	1255,9
5.	n = 100000	105,1	1589,6
6.	n = 120000	109,7	1607,3
7.	n = 140000	118,5	1652,4
8.	n = 160000	113,6	1625,6
9.	n = 180000	121,1	1619,9
10.	n = 200000	120	1782



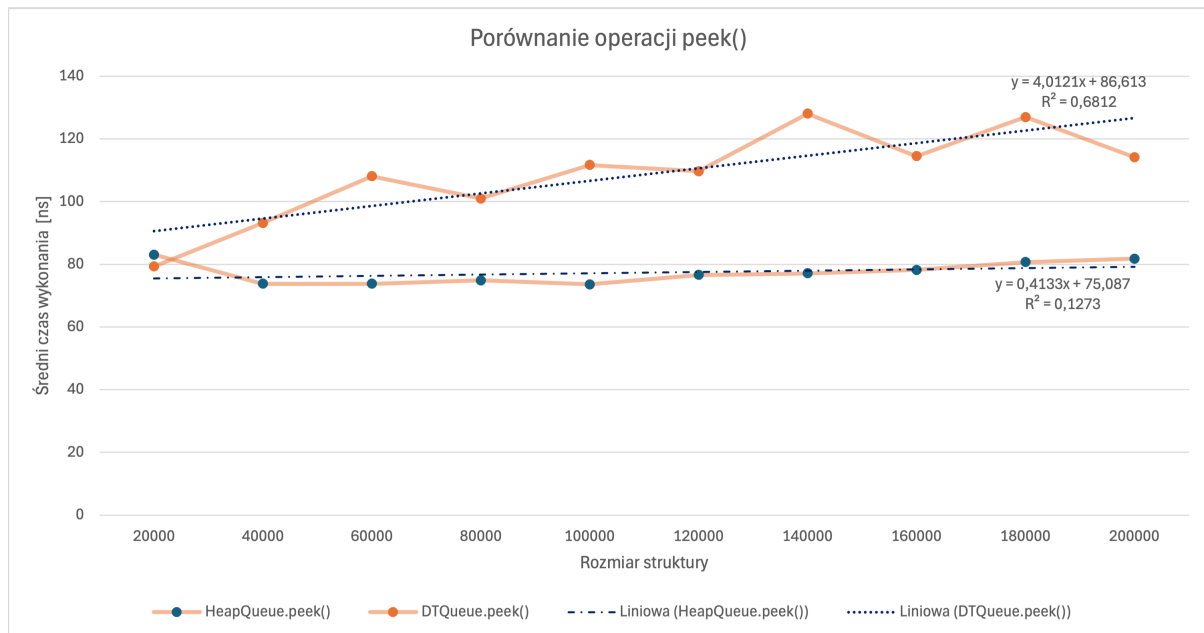
Rysunek 5: Zależność średniego czasu wykonania operacji `extractMax()` od rozmiaru struktury danych.

3.3 Operacja peek()

Poniżej zawarte zostało zestawienie złożoności obliczeniowych operacji `peek()`, dla różnych implementacji kolejki priorytetowej, oraz różnych zestawów danych wejściowych. Wyniki zostały zaprezentowane w postaci tabeli, oraz wykresu:

Tabela 5: Wyniki testów operacji `peek()`

Lp.	Size	Operation time [ns]	
		DTQueue	HeapQueue
1.	n = 20000	79,3	83,1
2.	n = 40000	93,2	73,8
3.	n = 60000	108,1	73,8
4.	n = 80000	101	74,9
5.	n = 100000	111,7	73,6
6.	n = 120000	109,7	76,6
7.	n = 140000	128,1	77,1
8.	n = 160000	114,5	78,2
9.	n = 180000	127	80,7
10.	n = 200000	114,2	81,8



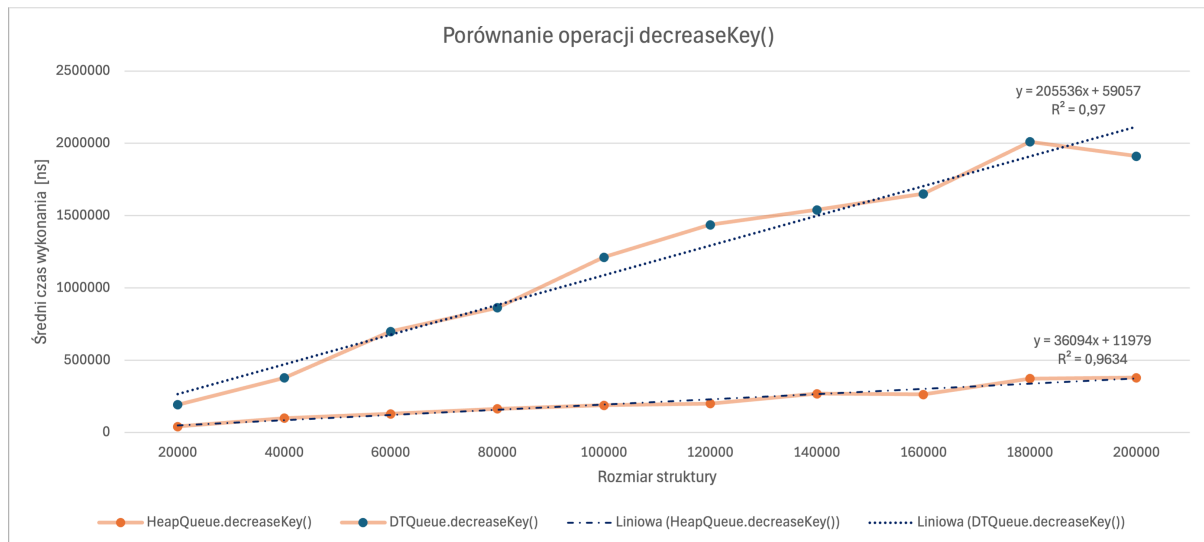
Rysunek 6: Zależność średniego czasu wykonania operacji `peek()` od rozmiaru struktury danych.

3.4 Operacja decreaseKey()

Poniżej zawarte zostało zestawienie złożoności obliczeniowych operacji `decreaseKey(item, newPrio)`, dla różnych implementacji kolejki priorytetowej, oraz różnych zestawów danych wejściowych. Wyniki zostały zaprezentowane w postaci tabeli, oraz wykresu:

Tabela 6: Wyniki testów operacji `decreaseKey()`

Lp.	Size	Operation time [ns]	
		DTQueue	HeapQueue
1.	n = 20000	192339	41064,5
2.	n = 40000	377470	99580,7
3.	n = 60000	699251	128405
4.	n = 80000	863270	163944
5.	n = 100000	1213000	188638
6.	n = 120000	1437070	199761
7.	n = 140000	1540090	268616
8.	n = 160000	1650860	262387
9.	n = 180000	2010520	372906
10.	n = 200000	1911170	379632



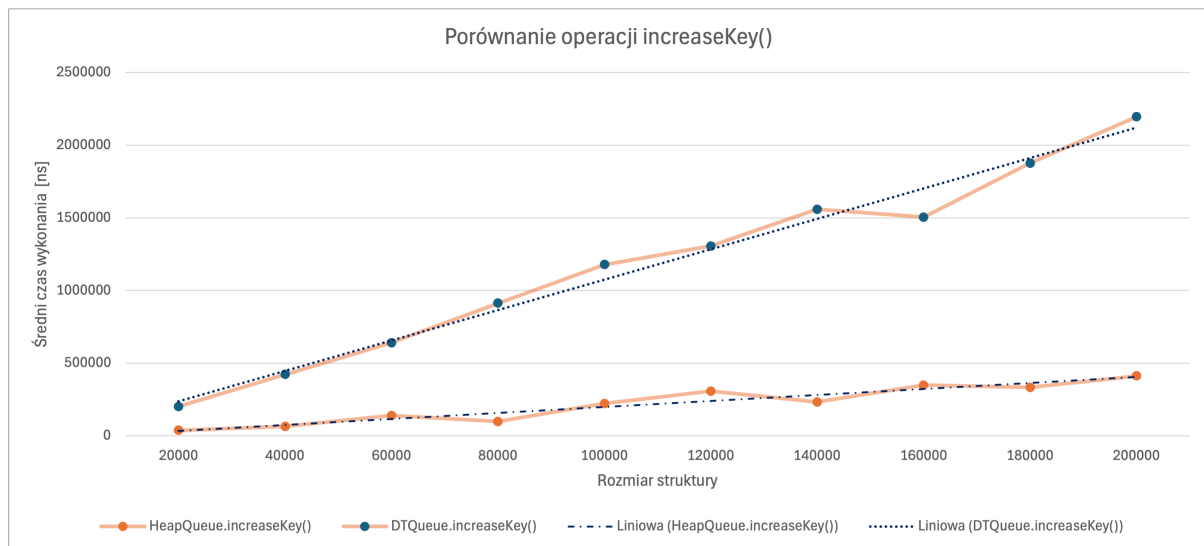
Rysunek 7: Zależność średniego czasu wykonania operacji `decreaseKey()` od rozmiaru struktury danych.

3.5 Operacja increaseKey()

Poniżej zawarte zostało zestawienie złożoności obliczeniowych operacji `increaseKey(item, newPrio)`, dla różnych implementacji kolejki priorytetowej, oraz różnych zestawów danych wejściowych. Wyniki zostały zaprezentowane w postaci tabeli, oraz wykresu:

Tabela 7: Wyniki testów operacji `increaseKey()`

Lp.	Size	Operation time [ns]	
		DTQueue	HeapQueue
1.	n = 20000	201844	37691
2.	n = 40000	421799	64038
3.	n = 60000	639475	138914
4.	n = 80000	912668	98733,2
5.	n = 100000	1178500	221995
6.	n = 120000	1305730	307469
7.	n = 140000	1558290	232876
8.	n = 160000	1504920	348800
9.	n = 180000	1876540	333168
10.	n = 200000	2195460	412540



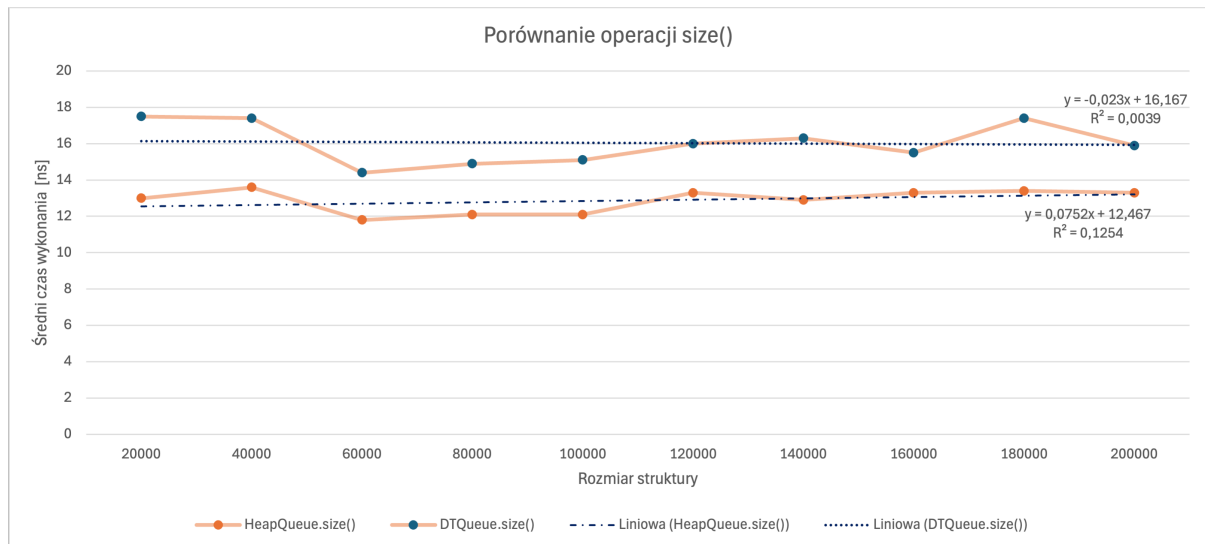
Rysunek 8: Zależność średniego czasu wykonania operacji `increaseKey()` od rozmiaru struktury danych.

3.6 Operacja size()

Poniżej zawarte zostało zestawienie złożoności obliczeniowych operacji `size()`, dla różnych implementacji kolejki priorytetowej, oraz różnych zestawów danych wejściowych. Wyniki zostały zaprezentowane w postaci tabeli, oraz wykresu:

Tabela 8: Wyniki testów operacji `size()`

Lp.	Size	Operation time [ns]	
		DTQueue	HeapQueue
1.	n = 20000	17,5	13
2.	n = 40000	17,4	13,6
3.	n = 60000	14,4	11,8
4.	n = 80000	14,9	12,1
5.	n = 100000	15,1	12,1
6.	n = 120000	16	13,3
7.	n = 140000	16,3	12,9
8.	n = 160000	15,5	13,3
9.	n = 180000	17,4	13,4
10.	n = 200000	15,9	13,3



Rysunek 9: Zależność średniego czasu wykonania operacji `size()` od rozmiaru struktury danych.

4 Wnioski

Przeprowadzone badania pozwoliły empirycznie zweryfikować teoretyczne złożoności obliczeniowe operacji kolejki priorytetowej zaimplementowanej na posortowanej liście (DTQueue) oraz kopcu binarnym (HeapQueue).

Operacja `enqueue()` jest kluczowym wyróżnikiem obu implementacji. DTQueue wymaga liniowego czasu $O(n)$ na znalezienie właściwej pozycji wstawiania, co przy $n = 200000$ skutkuje czasem ponad 1 ms. HeapQueue wykonuje tę operację w czasie $O(\log n)$, osiągając kilkaset nanosekund niezależnie od rozmiaru, jest to przewaga rzędu trzech rzędów wielkości. Wynika to z faktu, że DTQueue musi za każdym razem przejść przez istniejące węzły listy w celu znalezienia właściwego miejsca wstawienia, podczas gdy kopiec wykonuje jedynie porównania wzdłuż jednej ścieżki o długości $O(\log n)$.

Operacja `extractMax()` faworyzuje DTQueue: posortowana lista daje stały dostęp $O(1)$ do maksimum (≈ 100 ns), podczas gdy kopiec wymaga przesiewania $O(\log n)$ ($\approx 1000 - 1800$ ns). Jest to klasyczny kompromis projektowy między `enqueue()` a `extractMax()`. Operacje `peek()` i `size()` są stałe $O(1)$ dla obu struktur i osiągają porównywalne, bardzo krótkie czasy (poniżej 200 ns). Nie stanowią one różnicującego kryterium wyboru implementacji.

Operacje `decreaseKey()` i `increaseKey()` okazały się kosztowne dla obu struktur ze względu na konieczność liniowego wyszukania elementu $O(n)$. HeapQueue jest jednak konsekwentnie szybsza dzięki mniejszej stałej i efektywniejszej naprawie struktury po znalezieniu elementu. Wąskim gardłem jest tutaj wyszukiwanie elementu, kopiec nie jest strukturą przeszukiwalną, co wymusza liniowe skanowanie tablicy przed właściwą naprawą struktury.

Podsumowując: wybór implementacji zależy od charakteru obciążenia, jeśli dominuje `extractMax()` przy rzadkich wstawianiach wtedy DTQueue może być korzystniejsza. Jeśli dominuje `enqueue()` lub wymagana jest ogólna wydajność to HeapQueue jest zdecydowanie lepszym wyborem.